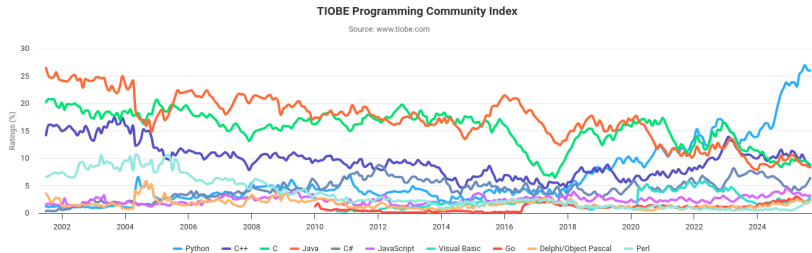


About language “popularity”



There are only two kinds of languages: the ones people complain about and the ones nobody uses. (Bjarne Stroustrup, the C++ programming language)

For scientific programming, a recommended choice today is C/C++ and/or Python. Development in Python is faster than in C/C++ but doesn't scale well unless it is used as glue language to interface between existing compiled code with a C API.

Compatibility between C and C++

It is almost but not completely true that C is a subset of C++. When (re)using existing C code into a C++ project you'll probably only need to take into account the following two :

- ▶ C++ has stricter rules for implicit type conversions.
- ▶ C++ function names are “mangled” in compiled object files.

We next describe how to take care of these incompatibility; the reasons behind these will become clearer later when we describe new features allowed by C++.

Turning C code into C++ code

I. At the level of source code : stricter conversion rules.

Contrary to C, in C++ a (void *) pointer cannot be implicitly cast into other pointer types. An important implication concerns malloc : code like e.g.

```
#include <stdlib.h>
int *p = malloc(10 * sizeof(int));
```

should be adapted as

```
#include <stdlib.h>
int *p = (int *)malloc(10 * sizeof(int));
```

or the more C++-ish

```
#include <cstdlib>
int *p = static_cast<int *>(malloc(10 * sizeof(int)));
```

It is common to use the equivalent

```
#include <cstdlib>
int *p = static_cast<typeof(p)>(malloc(10 * sizeof(*p)));
```

so that if one changes mind later and opt for e.g. float instead of int then the change needs only be made in one place.

Turning C code into C++ code

Note:

Later we shall see that one could use the `new` operator instead of the `malloc` function.

```
int *p = new int[10];
```

In such a case the call to `free` will be replaced by the `delete` operator:

```
delete [] p;
```

The `new` and `delete` operators are non existing in C.

For types more complex than fundamental types (more precisely: POD types, for Plain Old Data) they have more implications than just memory allocation (call of constructors and destructors in particular), so they should not be considered equivalent.

Besides, there is no native C++ replacement for `realloc`.

Turning C code into C++

II. Inside header files : avoiding name mangling.

If you intend to share your compiled code in the form of object files or shared libraries in such a way that it can be used with any language with a C API call capability (many languages do), you should modify your headers in two ways. First they should not contain any C++ feature not available in C, and second a safe guard (extern C) should be used to prevent from name mangling by the C++ compiler.

```
#pragma once
/* Includes go first as usual */
.
.
.
/* Then C interface statement */
#ifdef __cplusplus
extern "C" {
#endif
/* The bulk of the header goes here */
.
.
.
/* End of C interface */
#ifdef __cplusplus
}
#endif
```

Turning C code into C++ code

III. At the level of the compiler :

You should use a C++ aware compiler...

Inside the GNU family, g++ should replace gcc.

It is actually not a bad idea to use g++ even for C code. Since C++ rules regarding types and implicit conversions are stricter, it can often catch bugs that you may not have noticed otherwise. Turning on all warnings and debug symbols is recommended in debug builds :

```
g++ -Wall -g [...]
```

For release builds instead, debug symbols and assertions check should be turned off, and compiler optimization turned on:

```
g++ -DNDEBUG -O3 [...]
```

If you end-up using CMake (recommended), the distinction is done using either

```
cmake -DCMAKE_BUILD_TYPE=Debug [...]
```

or

```
cmake -DCMAKE_BUILD_TYPE=Release [...]
```

C++ features: References

References look and taste like pointers, but they are not pointers.

The main differences are :

- ▶ A reference to a variable must always be valid (i.e. there is no equivalent of a NULL pointer), and in particular it must be initialized by the same time it is declared.
- ▶ A reference cannot be reassigned to refer to another variable than the one it was initialized with. Also there is no such thing as reference arithmetics, contrary to pointer arithmetics.
- ▶ A reference can later be used in code exactly as if it was the original variable (whereas pointers need to be dereferenced, they have special operators for field access (-> instead of .).
- ▶ References are introduced by the symbol & rather than *.

They can be understood as *aliases* to existing variables.

```
int a;  
a = 5;  
int &b; /* Syntax error: must be initialized as soon as declared */  
int &c = a; /* OK, c is a reference to a */  
c = 7;  
printf("Value of a : %d\n", a); /* -> Value of a : 7 */
```

C++ features: References

By far the main use of references is *pass by reference* of function arguments. Indeed, recall that C arguments passing is always by copy.

Example:

```
int *build_adjacency_table(const struct Mesh& m)
{
    int *adj = (int *)malloc(3 * m.tri * sizeof(int));
    /* Note m.tri, not m->tri */
    [...]
    return adj;
}
```

In the above, it is said that the argument *m* is a reference to a (constant) struct Mesh.

In code it could then be used as in

```
struct Mesh m;
[...]
int *adj = build_adjacency_table(m); /* Note: m, not &m */
```

C++ features: Functions overloading

In C, a function is characterized (i.e. uniquely determined) by its name.

In particular, in C it is not possible to use the same function name for, e.g., taking the absolute value of a double and the absolute value of an int. For that reason, the standard C library has both the functions

```
int abs(int n);
```

and

```
double fabs(double x);
```

It is somewhat inconvenient because:

1. it is a limitation of expressivity of the language,
2. it is a possible source of bugs:

```
double a = -0.5;  
double b = abs(a);
```

This code will compile and the compiler will produce no warning (be it gcc or g++), even with `-Wall`. But the resulting value of `b` is 0, not 0.5. This is not a bug, but a consequence of implicit conversions for arithmetic types.

C++ features: Functions overloading

Instead, in C++ a function is characterized by :

1. its name,
2. the number of its arguments,
3. the types of its arguments.

Note : Only the types of arguments, **not** the type of its return value.

As a consequence, it is valid to declare:

```
int abs(int n);  
double abs(double x);
```

Note that the implementation of both would be identical except for the types.
Templates (see later) are precisely meant for such situations:

```
template <typename T>  
T abs(T n)  
{  
    return n < 0 ? -n : n;  
}
```

C++ features: Functions overloading

In the same vein, C++ functions can also have *default arguments*.

A function declared as:

```
int read_mesh(const char* filename, bool check_integrity = false);
```

could be called either as

```
read_mesh(filename);
```

(in which case `check_integrity` would be `false`), or e.g. as

```
read_mesh_file(filename, true);
```

in which case it would be `true`. Useful when the function is often called with the default value, and rarely with a different value. Use with parcimony.

There may be more than one argument with a default value, but if there are N of them they must be the last N arguments of the function.

Note: name mangling in object files by the C++ compiler is a consequence of this function overload feature. A compatible C API should therefore never use that feature at the level of the interface (it can use it internally though).

C++ features: Methods

In C++ a method is a function that puts a special emphasize on one of its argument, of struct type. It corresponds to a shift of view point typical of object oriented programming: objects running methods, instead of functions applied to objetcs.

Example :

```
struct Mesh {  
    /* Fields */  
    int nvert;  
    int ntri;  
    struct Vertex *vertices;  
    struct Triangle *triangles;  
    /* Methods declaration */  
    int read_from_file(const char* filename);  
    float compute_area() const;  
};
```

which could then be used as in

```
struct Mesh m;  
m.read_from_file("test.mesh");  
double area = m.compute_area();
```

C++ features: Methods

Methods have direct access to the data (and also other methods) of their associated struct. In the previous example the second method could be defined (it was already declared) as

```
double Mesh::compute_area() const
{
    double area = 0;
    for (int i = 0; i < ntri; ++i)
    {
        struct Vertex v1 = vertices[triangles[i].v1];
        struct Vertex v2 = vertices[triangles[i].v2];
        struct Vertex v3 = vertices[triangles[i].v3];
        area += triangle_area(v1, v2, v3);
    }
    return area;
}
```

where `triangle_area` would be a utility function already defined elsewhere.

The operator `::` above is called the *resolution* operator. Here it means : the method `compute_area` of the struct `Mesh`.

The *free function* equivalent prototype (a free function is a C++ way to refer to a function which is not a method) would be

```
double compute_mesh_area(const struct Mesh &m);
```

C++ features: Constructors and Destructors

Constructors (resp. destructors) are special methods that are implicitly (if not explicitly) called when an instance of a struct enters (resp. leaves) the current scope in the code.

They are typically used to initialize the struct fields or process with some memory allocation, for constructors, and for clean-up, for destructors.

They must have special names: constructors are named identically to their corresponding struct name, and destructors names are the struct name with a `~` in front.

(Following code refer to the [didiersmets/Myosotis](#) github repo)

Example : (show and comment `include/array.h` and then `include/hash_table.h`)

C++ features: Operators overloading

(show and comment include/vec3.h)

C++ features: Derived structs and inheritance

(show and comment include/vertex_table.h)